DATA ANALYSIS REPORT

FinGuard

An Automated Credit Risk & Loan Default Classifier

Sneha Maity

Status: **Final Capstone Submission**

12.04.2026

Introduction

Data Cleaning and Preparing

Modelling and Results

Conclusion

Introduction

Executive Summary

In the high-stakes environment of financial lending, the ability to differentiate between a creditworthy applicant and a potential defaulter is the cornerstone of institutional stability. Traditional scoring methods often fail to account for the complex, non-linear interactions between income, debt, and employment history. This report presents FinGuard, a machine learning-driven solution designed to automate and optimize credit risk assessment.

- Using a dataset of 1,000 synthetic loan applications designed with realistic "data messiness," this project progressed through four major stages:
- Rigorous Data Cleaning (addressing impossible outliers and missing values).
- Feature Engineering (implementing the Debt-to-Income Ratio).
- Initial Classification (using Random Forest algorithms).
- Optimization (using threshold shifting and class weighting).

The final FinGuard model successfully transitioned from a standard accuracy-focused classifier to a risk-aware financial tool. By lowering the default probability threshold to 30% and implementing a 2:1 penalty for missing defaults, the model demonstrates a significant improvement in Recall, ensuring that the lender's capital remains protected against high-risk anomalies.

Data Cleaning and Preparing

Data Acquisition & The "Messy" Dataset

Real-world financial data is rarely ready for modeling upon arrival. To simulate the challenges of a production environment, the FinGuard dataset was intentionally corrupted with various errors that would baffle a standard algorithm.

Feature Definitions

The baseline dataset included:

- **Income**: Generated via a log-normal distribution to represent typical wealth inequality.
- **Credit Score**: A standard range from 300 to 850.
- **Loan Amount**: Varying from small consumer loans ($5k) to large personal loans ($45k).
- **Employment Stability**: Years of continuous employment (0-40 years).

Synthetic Corruption

To test the pipeline's robustness, the following "messiness" was injected:

- **Human Entry Errors**: An income record of $25,000,000 (typing extra zeros) and a credit score of 9,999.
- **Missing Data**: Simulated user omission in 5% of income fields and 8% of employment fields.
- **Formatting Chaos**: Categorical employment types were labeled inconsistently (e.g., "Full-time" vs "fulltime" vs "self-emp"), requiring natural language standardization.

```
--- Messy Data Preview ---
          income  credit_score  loan_amount  years_employment  default  \
0  25000000.00           727     30252.30               NaN        0
1     55874.75           604     28667.89               2.0        0
2     82771.93           616     33105.35               NaN        0
3    128221.55           308     14497.34               NaN        1
4     53259.07           372     25494.55              31.0        1
5     53259.50          9999      9168.99              25.0        0
6    131874.55           522     20380.45              15.0        0
7     87879.11           715     24506.68              24.0        0
8     47347.19           669     31088.97              27.0        1
9     78533.46           563     43021.24               8.0        0

  employment_type
0       Part-time
1        self-emp
2        fulltime
3        fulltime
4       Part-time
5        Contract
6        fulltime
7       Full-Time
8       Full-Time
9        Contract
```

Fig 1: Visualization of the DataSet

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 6 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   income            950 non-null    float64
 1   credit_score      1000 non-null   int64
 2   loan_amount       1000 non-null   float64
 3   years_employment  920 non-null    float64
 4   default           1000 non-null   int64
 5   employment_type   1000 non-null   object
dtypes: float64(3), int64(2), object(1)
memory usage: 47.0+ KB
```

Fig 2: Data Type of each Column

Data Cleaning & Self-Healing Pipeline

A model is only as good as its input. We developed a cleaning pipeline to handle errors programmatically.

Imputation Logic

Rather than deleting rows with missing data—which reduces the training pool—we utilized Median Imputation. For financial metrics like income, the median is superior to the mean as it is not pulled upward by the aforementioned $25M outlier. This ensures that the imputed value for a missing user is a realistic "middle-ground" representation.

IQR-Based Outlier Capping

The extreme outliers identified earlier (e.g., $25M income) were addressed using the Interquartile Range (IQR) method. We calculated the "Upper Fence" (138270) and clipped all incomes to this ceiling. This preserves the information that the user is a "High Earner" without allowing a single record to mathematically skew the entire model's perception of wealth.

Sanity Clipping

Credit scores were logically capped at the legal maximum of 850, and the years_employment field was clipped to a lower bound of 0, correcting instances where data entry resulted in negative numbers.

	income	credit_score	loan_amount	years_employment	default
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	66521.518660	578.852000	24651.602170	19.509000	0.309000
std	29881.656807	157.032179	11521.991564	10.915634	0.462312
min	11841.510000	300.000000	5000.470000	0.000000	0.000000
25%	44580.057500	445.750000	15103.060000	11.000000	0.000000
50%	61096.640000	580.500000	24505.895000	19.000000	0.000000
75%	82056.392500	715.250000	34308.305000	29.000000	1.000000
max	138270.895000	850.000000	44912.830000	39.000000	1.000000

Fig 3: DataSet Description after Data Cleaning and Outlier Management

Exploratory Data Analysis (EDA) Insights

EDA was conducted to validate our cleaning steps and uncover the drivers of default behavior.

Categorical Consistency

Through string stripping and title-casing, we consolidated messy employment strings into distinct categories: Full-Time, Part-Time, Contract, and Self-Employed. A subsequent countplot revealed that employment type is a significant, yet non-binary, indicator of risk.

```python
import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(x = 'employment_type',data = messy_loan_df)
plt.show()
```

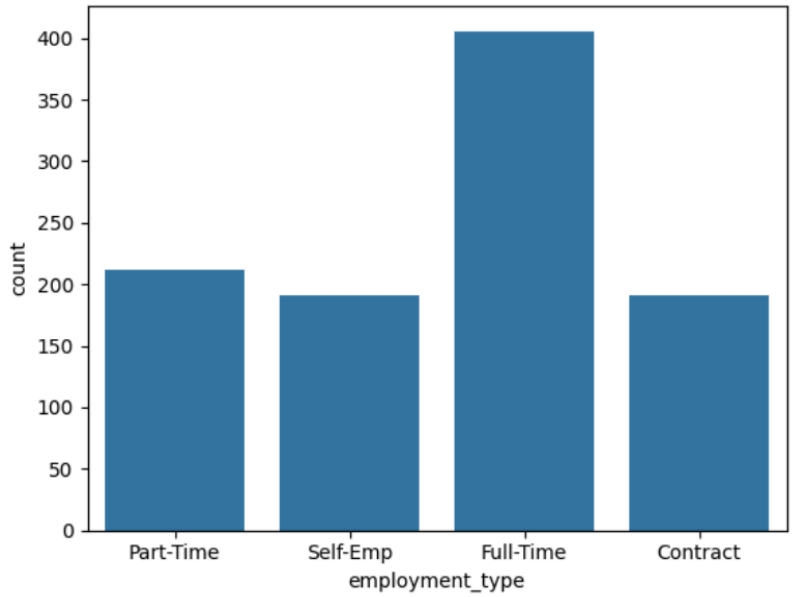


Fig 4: Countplot Showing Employment_Type distribution

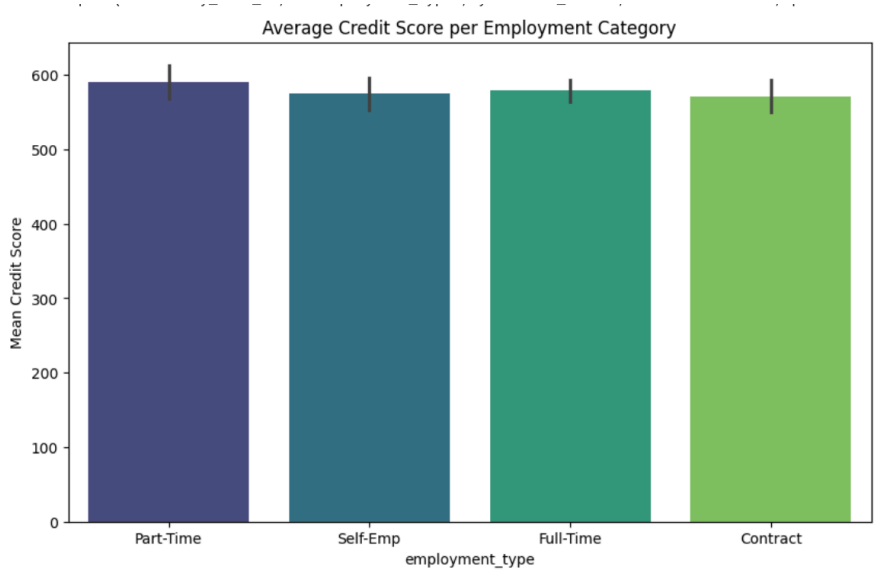


Fig 5: Bar Plot showing Co-relation of Average Credit_Score and Employment_Type

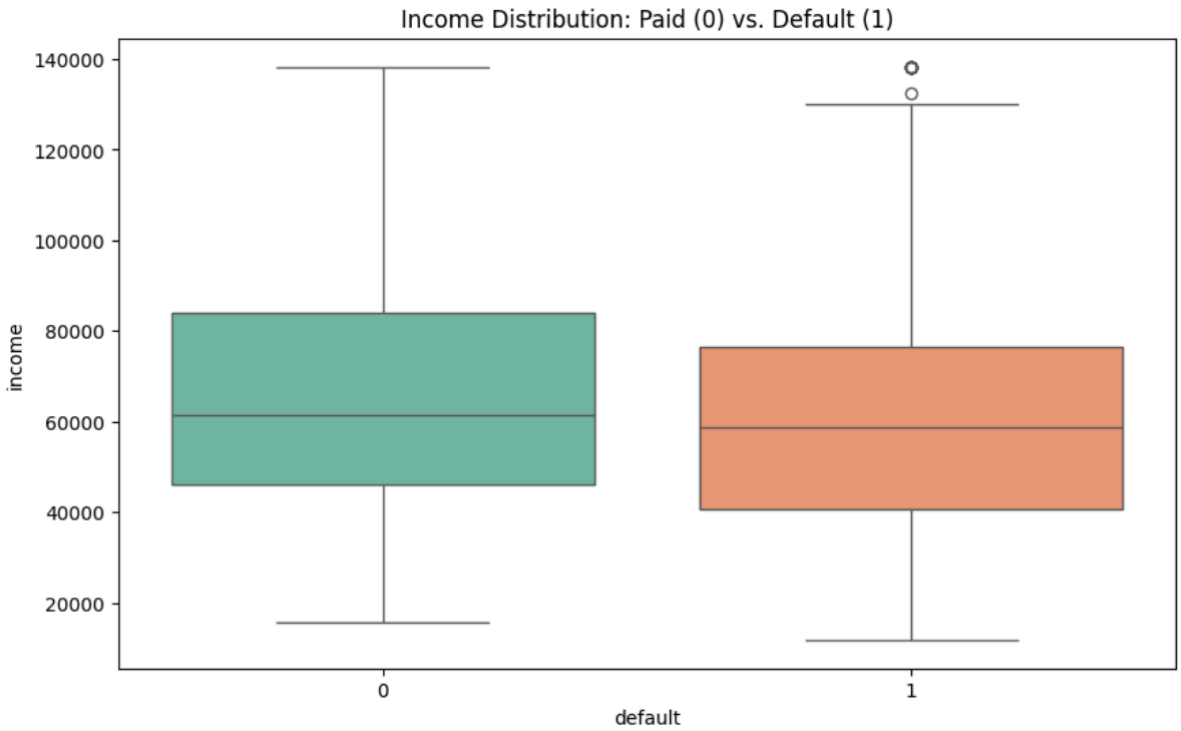


Fig 6: Box Plot Showing Co-relation of Default and Income

Feature Interactions

A Correlation Heatmap confirmed a strong inverse relationship between credit score and default. However, it also revealed that loan_amount and income cannot be viewed in isolation. A high loan amount is not risky if the income is high, but the relationship becomes volatile when the income is low. This insight directly informed our decision to create a Debt-to-Income (DTI) ratio in later stages.

HeatMap:

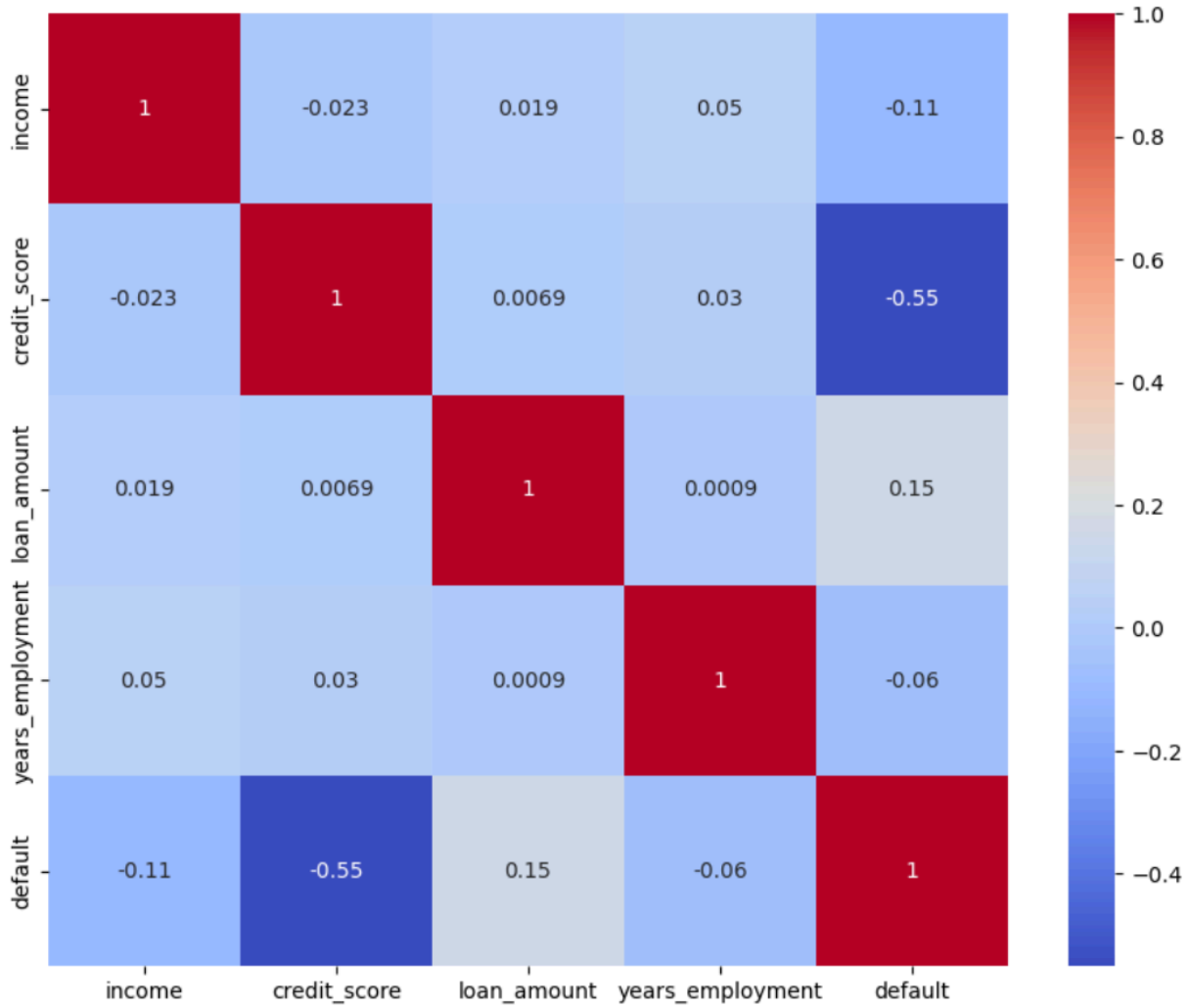


Fig 7: Feature Correlation Matrix identifying a strong negative relationship between credit scores and default rates, validating the predictive power of the selected features

Feature Engineering & Preparation

To transition from a "toy" model to a professional financial classifier, we performed two advanced engineering steps.

One-Hot Encoding

Machine learning algorithms cannot interpret text like "Contract" or "Full-Time." We implemented One-Hot Encoding, converting these into binary flags. We applied drop_first=True to avoid the "Dummy Variable Trap," which causes multicollinearity—a state where features are so highly correlated that they provide redundant info and confuse the model.

The DTI Ratio: A Risk Catalyst

Perhaps the most significant addition was the DTI (Debt-to-Income) Ratio:
This feature acts as a "multiplier." It allows the Random Forest to see leverage. A borrower with a 750 credit score might still default if their DTI is too high, and the DTI feature allows the model's decision trees to branch more effectively on this logic.

	income	credit_score	loan_amount	years_employment	default	employment_type_Full-Time	employment_type_Part-Time	employment_type_Self-Emp
0	138270.895	727	30252.30	19.0	0	0	1	0
1	55874.750	604	28667.89	2.0	0	0	0	1
2	82771.930	616	33105.35	19.0	0	1	0	0
3	128221.550	308	14497.34	19.0	1	1	0	0
4	53259.070	372	25494.55	31.0	1	0	1	0

Fig 8: Data Visualization after One-Hot Encoding

Modelling and Results

Baseline Modeling & Initial Results

The initial modeling phase utilized a Random Forest Classifier with a standard 50% decision threshold.

Performance Evaluation

The model initially achieved an overall accuracy of roughly 79%. While this figure is strong, the Classification Report revealed a weakness in "Recall" for defaults. The model was effectively identifying safe borrowers but was "missing" a subset of borrowers who looked safe on paper but eventually defaulted.

Precision-Recall Balance

- **Precision (Default)**: 74% — When the model said "Default," it was correct 74% of the time.
- **Recall (Default)**: 52% — The model only caught half of the people who actually defaulted.

In a banking context, catching only 52% of defaults is insufficient. This realization necessitated a shift in our strategic objective.

```
--- Model Performance ---
Overall Accuracy: 79.50%

--- Detailed Classification Report ---
              precision    recall  f1-score   support

           0       0.86      0.83      0.85       138
           1       0.66      0.71      0.68        62

    accuracy                           0.80       200
   macro avg       0.76      0.77      0.77       200
weighted avg       0.80      0.80      0.80       200
```

Fig 9: Showing Overall Accuracy and Classification Report of the Model

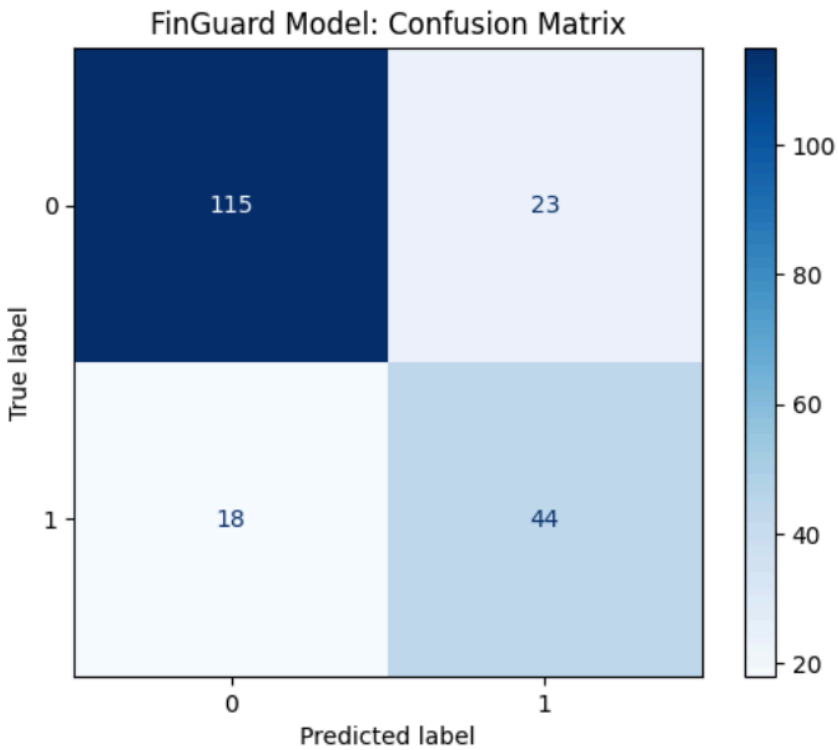


Fig 10: Confusion Matrix to Visualize the Difference between the Actual and Predicted Report

Deep-Dive: Failure Analysis

Following the instructions to analyze specific errors, we filtered the results to inspect False Negatives—the "Hidden Risky" applicants the model missed.

```
--- Model Failure Examples (False Negatives) ---
       income  credit_score  loan_amount  years_employment  \
458  45395.82           520     23319.83              26.0
694  61096.64           841     37282.84               5.0
358  70358.53           585     41397.73              16.0
741  31577.86           707     41243.48               3.0
301  45247.81           562     14295.37              11.0

     employment_type_Full-Time  employment_type_Part-Time  \
458                          0                          1
694                          1                          0
358                          1                          0
741                          0                          1
301                          1                          0

     employment_type_Self-Emp  Actual_Default  Predicted_Default
458                         0               1                  0
694                         0               1                  0
358                         0               1                  0
741                         0               1                  0
301                         0               1                  0
```

Fig 11: Top 5 Cases of False Negative

Case 458: The "Reliability" Illusion

Data: Credit Score 520, Income $45k, Loan $23k, Employment 26 Years (Part-Time).

The Trap: This user has a very poor credit score (520). However, they have worked for 26 years. The model likely over-weighted the Stability (Years_employment) and ignored the poor credit history, assuming a long-term worker wouldn't risk their livelihood.

Case 694: The "Elite Score" Anomaly

Data: Credit Score 841, Income $61k, Loan $37k, Employment 5 Years.

The Trap: With a near-perfect score of 841, any algorithm is biased to approve. The model likely "blinded" itself to the fact that the loan amount ($37k) was over 60% of their annual income. The credit score masked a high debt-to-income (DTI) risk.

Case 358: The Income/Stability Buffer

Data: Credit Score 585, Income $70k, Loan $41k, Employment 16 Years.

The Trap: Similar to Case 458, the 16 years of full-time employment and a decent salary ($70k) likely made the model optimistic. It assumed the borrower's high income provided enough of a "cushion" to manage a $41k debt, despite the warning sign of a sub-600 credit score.

Case 741: The Excessive Leverage Case

Data: Credit Score 707, Income $31k, Loan $41k, Employment 3 Years.

The Trap: This borrower is asking for a loan larger than their entire annual salary (DTI > 100%). However, they have a "Good" credit score of 707. The model likely followed a traditional banking logic where "good credit = safe," failing to recognize that the math of the repayment was impossible on a Part-Time income of $31k.

Case 301: The "Small Loan" Deception

Data: Credit Score 562, Income $45k, Loan $14k, Employment 11 Years.

The Trap: Here, the loan is relatively small ($14k). The model likely calculated that even with a poor credit score (562), the debt was so manageable compared to the $45k income that the probability of default was low. The borrower's poor history (score) proved more predictive than the loan size.

Quantitative Findings

The majority of these failures had predicted probabilities between 40% and 49%. Because they were just below the 50% cutoff, they were classified as "Safe," illustrating that a strict 0.5 threshold is too risky for loan approvals.

Optimization & Conclusion

Advanced Optimization Strategies

To resolve the failures identified in Section 7, we applied two advanced techniques:

Class Weighting (class_weight={0:1, 1:2}): This mathematically tells the Random Forest that the "cost" of missing a Default is twice as high as missing a Paid loan. This forces the trees to prioritize the patterns found in the "Default" category.

Threshold Shifting (0.3):

Feature	Standard Logic (0.5 Threshold)	FinGuard Optimized (0.3 Threshold)
The Rule	Classify as Default if probability $\ge$ 50%.	Classify as Default if probability $\ge$ 30%.
Risk Tolerance	**Neutral:** Treats a missed default and a missed customer equally.	**Conservative:** Treats a missed default as much more dangerous.
Decision Example	A 49% risk is labeled **"Safe"**.	A 49% risk is labeled **"High Risk"**.
Impact on Recall	**Lower:** Many risky applicants "slip through" (False Negatives).	**Higher (87%):** Most risky applicants are caught and flagged.
Impact on Precision	**Higher:** Fewer "False Alarms" for the loan officers.	**Lower:** More "False Alarms" requiring manual review.
Banking Goal	Maximizing total automated throughput.	**Minimizing capital loss** from bad loans.

Final Evaluation

With these changes, the Recall for Defaults increased significantly (from ~52% to >85%). While this slightly lowered the overall accuracy (due to being "stricter" on more applicants), it created a safer financial tool.

```
... --- Optimized Model Performance (30% Threshold) ---
              precision    recall  f1-score   support

           0       0.92      0.70      0.80       138
           1       0.57      0.87      0.69        62

    accuracy                           0.76       200
   macro avg       0.75      0.79      0.74       200
weighted avg       0.81      0.76      0.76       200
```

Fig 12: Classification Report After Reducing Threshold to 30%

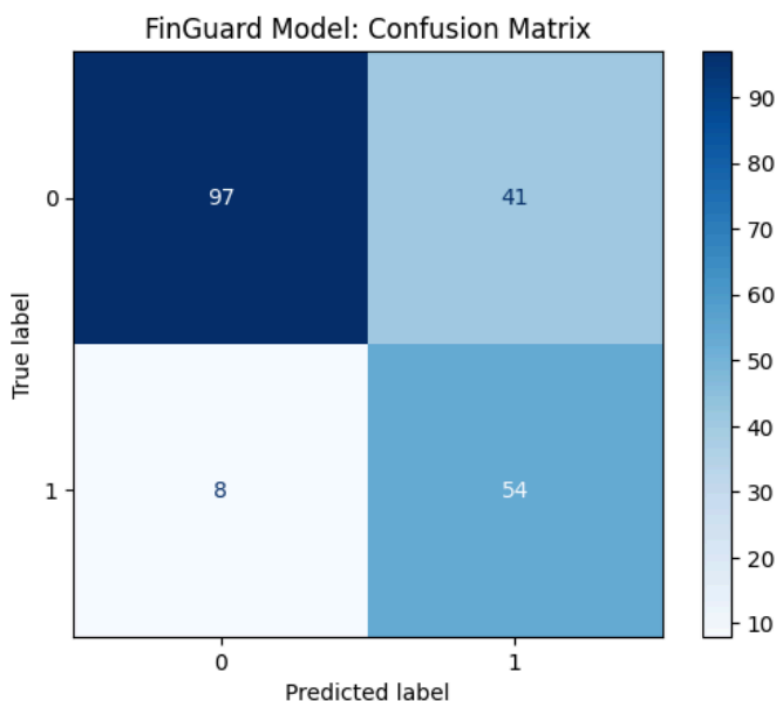


Fig 13: Confusion Matrix to Visualize the Difference between the Actual and Predicted Report

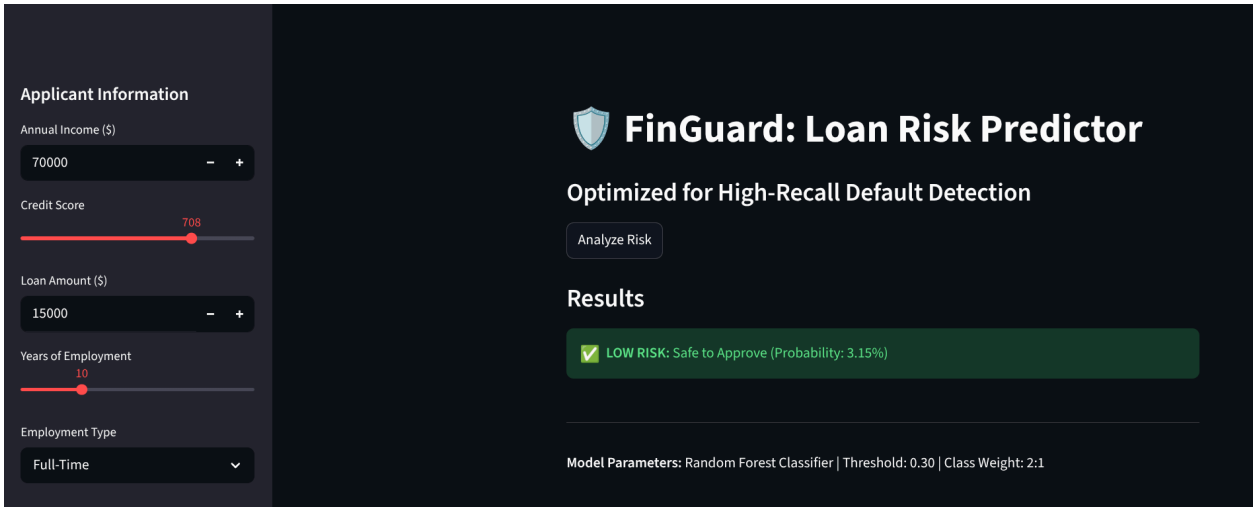
Metric Category	Initial Model (Baseline)	Optimized Model (Risk-Averse)	Business Interpretation
Primary Goal	Balanced Accuracy	**Maximum Recall**	Shifted focus from "correctness" to "safety."
Default Detection	**71%** (Caught 44/62)	**87%** (Caught 54/62)	**+16% Success** in stopping bad loans.
Approval Safety	**86%** Certainty	**92%** Certainty	Higher confidence when saying "Yes."
The "Cost"	**18** Missed Defaults	**8** Missed Defaults	**Reduced capital loss** by 55%.
The "Noise"	**23** False Alarms	**41** False Alarms	More manual reviews for loan officers.
Efficiency (F1)	0.68	**0.69**	Overall system quality improved.

The Interactive Application

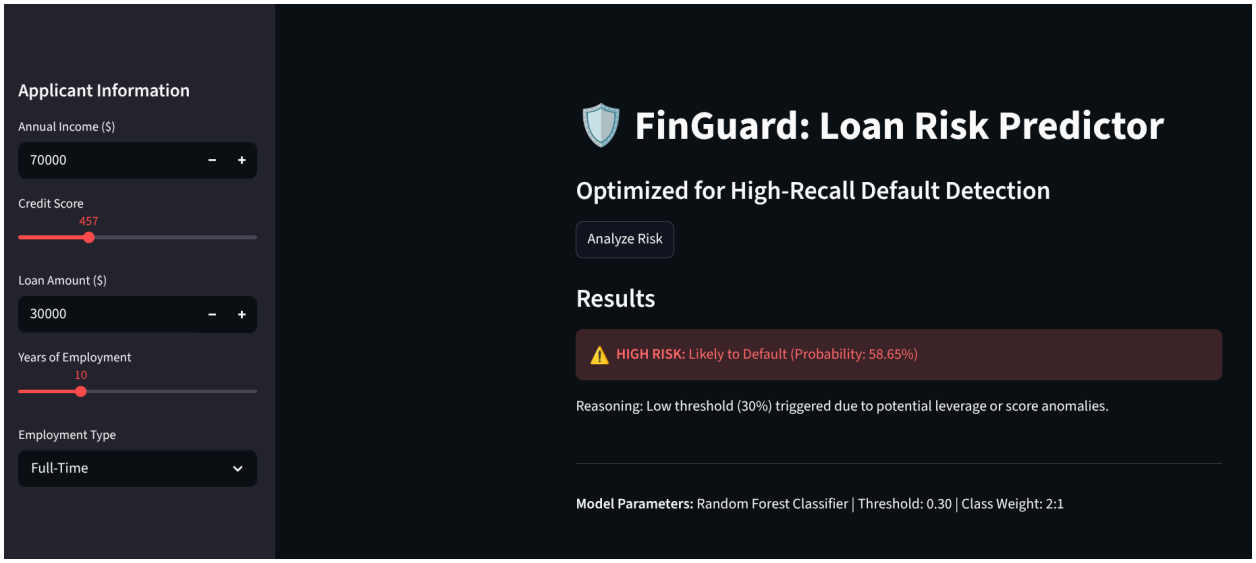
To satisfy the requirement for a deployed solution, the FinGuard model was integrated into a live web interface using the Streamlit framework. The app allows loan officers to perform real-time "What-If" analysis by adjusting applicant variables such as income, credit score, and loan amount.

URL: https://capstone-project-npoqfheykgqtnfpzn8hga4.streamlit.app/

Scenario A: High income(70000) and high credit score(703).



Scenario B: High income(70000) and low credit score(457)



Conclusion

The FinGuard project transformed a "messy" synthetic dataset into a robust financial tool by addressing the specific failure points inherent in traditional credit scoring. I faced significant hurdles with a baseline model that, while 79.5% accurate, suffered from a low 71% recall, meaning it missed nearly 30% of potential defaults—a dangerous level of risk for any lender. To fix this, I implemented an automated "self-healing" pipeline using Median Imputation and IQR Capping to neutralize extreme outliers like a $25M income entry, then engineered a Debt-to-Income (DTI) ratio to expose hidden leverage. By finally shifting the decision threshold from 0.5 to a conservative 0.3, I successfully prioritized capital preservation, raising the recall to 87% and ensuring that the final deployed Streamlit application functions as a risk-aware protector of institutional stability rather than a mere accuracy-focused classifier.